Railway Deadlock Avoidance in ASP

Based on "Avoiding Deadlocks in Synchronous Railway Simulations" (Pachl, 2007) [1]

Iannis Waldow

Institut für Informatik und Computational Science
Universität Potsdam

September 9, 2025

Outline

1. Introduction
2. Simulation Systems
3. Movement Consequence Analysis
4. Dynamic Route Reservation
5. Conclusion

What are Deadlocks?

In railway environments *deadlocks* refer to situations in which a set of trains cannot continue on their path because another train of the set is blocking it.

Once this blocking relationship between the trains of the set becomes circular there is no way to avoid the impending deadlock.

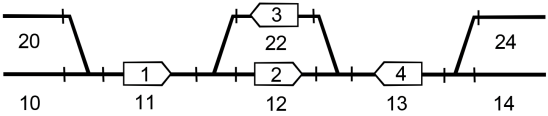


Figure: Example Deadlock Situation from the paper

When can deadlocks happen?

The paper highlights the four conditions that need to be true of the rail environment for a deadlock to possibly occur:

- "**Mutual Exclusion**": Two trains cannot occupy the same section of track at the same time.
- "**Wait For**": A train can wait on their current section until their next section is free.
- "**No Preemption**": A train cannot enter a section until another train occupying that sections has cleared it.
- "**Circular Wait**": It is possible for waiting trains to form closed circles, in which every train waits for the release of sections occupied by another train.

In operation, a timetable free of deadlocks is created beforehand.

As long as changes made due to the schedule don't affect the sequence of trains going through a section a deadlock will not occur.

We will look at the deadlocks in more detail within the domain of railway simulation.

There are two types of railway simulation systems:

- Asynchronous
- Synchronous

What are Asynchronous Simulation Systems?

Similar to scheduling in real operation, the idea is to separate scheduling from running the simulation.

In the scheduling step the process is generally to

- Create train paths according to scheduling rules
- Insert waiting for cases of conflict by priority

In the following running operation delays are introduced at random, which get resolved using rules that reflect “real dispatching”.

What are Synchronous Simulation Systems?

- Unlike their asynchronous counterpart, synchronous simulation systems purely simulate the **real time operation of the railway environment**
- A schedule can be provided from outside of the simulation system. But since scheduling and rescheduling are not a part of the simulation system itself **deadlocks can occur**
- The paper notes, that in most synchronous simulation systems only very simple deadlock prevention logic is present

---

The question: In this synchronous environment, how do we detect and prevent scheduling decisions that lead to deadlocks?

Movement Consequence Analysis or "MCA" is one of the two approaches brought forward by the paper to address deadlock avoidance.

The idea is to look at the necessary consequences of taking a given scheduling decision on the sequence of train movement in the railway system.

These consequences are separated into **primary** and **secondary** consequences.

Primary Consequences

There are two types of primary consequence of a move.

- **Condition**: The move results goes onto or over a cell occupied by another train
  - **Consequence**: The front train has to move up
- **Condition**: The move places the train on the path of an oncoming train and this oncoming train cannot be avoided later
  - **Consequence**: The oncoming train moves past first

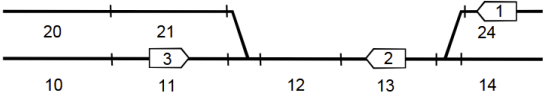
Secondary Consequences

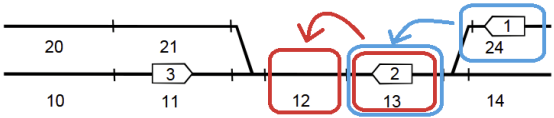
In primary consequences a train is caused to move as a consequence of *another* train's movement.

In secondary consequences a train is caused to moves as a consequence of its *own* movement.

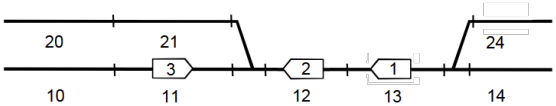
- **Condition**: A train moves onto a section of track, where it blocks the passing of oncoming trains
  - **Consequence**: The train moves until it isn't blocking any oncoming trains anymore

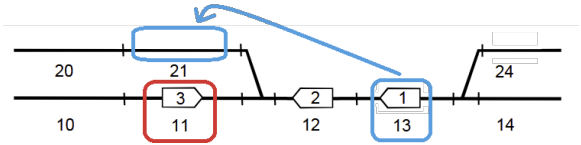
Of Consequence Types: Primary

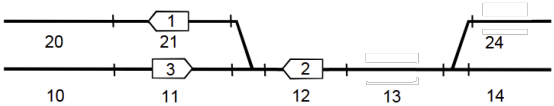


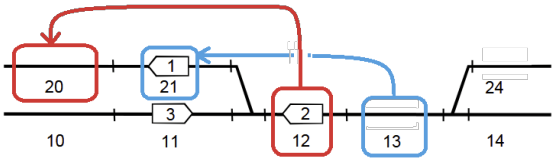


Of Consequence Types: Secondary






1
20
21
24
3
2
10
11
12
13
14


1
20
21
24
3
2
10
11
12
13
14

The main disadvantages of MCA are

- Train routes are fixed in advance!
- Hard to adapt into some simulation systems
  - In ASP: Logic hard to keep track of
- Not immediately clear on how to read and interpret

To address these problems another approach was developed: “Dynamic Route Reservation”.

The general idea of DRR is to have trains make reservations of multiple sections of track into the future.

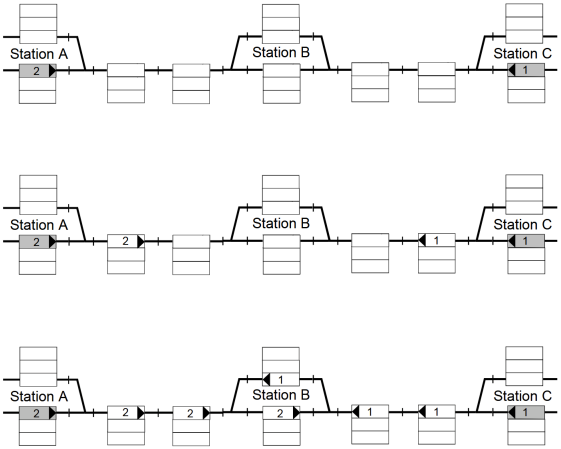
These reservations have to be made according to scheduling rules, which avoid deadlocks.

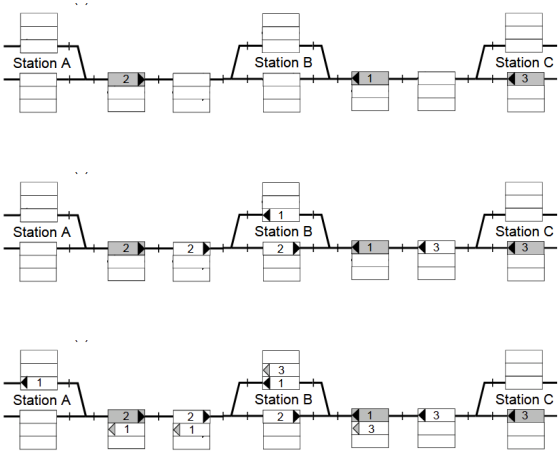
If a train's reservation for a section of track is valid according to the rules it gets added to the reservation queue for that section. A train can then only enter that section if it is at the front of the reservation queue.

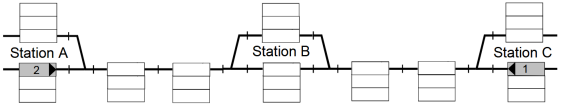
As a simplification, trains on bidirectional tracks will always continue until they are back on directional tracks. This simplifies secondary consequences from MCA.

The paper goes over the rules of trains making reservation of tracks. These are:

1. A train can only enter a track with a reservation
2. If a train reserves a track leading into a bidirectional track, this track too *must* be reserved
3. If there is already a reservation queue or another train present on the track, the reservation gets appended to that queue
4. If a reservation is added to a track's queue, the trains ahead in that queue *must* make reservations leaving the track
5. If a train tries to reserve a track that leads into another track occupied or reserved for an opposing train, the reservation fails
6. Rules 2 and 4 stipulate *necessary* reservations following an original reservation. If one of these reservations fail the original reservation also fails.


Station A
Station B
Station C
2
1
Station A
Station B
Station C
2
2
1
1
Station A
Station B
Station C
1
2
2
2
2
1
1
1


Station A
Station B
Station C
2
1
3
Station A
Station B
Station C
1
2
2
2
1
3
3
Station A
Station B
Station C
1
3
1
2
1
2
1
2
1
3
3
3



We translate the environment to ASP first:

```
section(10). section(11). section(12).
...
connected(section(10),section(11), east).
connected(section(11),section(12), east).
...
bidirectional(section(11)).
bidirectional(section(12)).
...
```

Continuing the environment encoding:

```
directional(section(10)).
directional(section(20)).
...

train(1). train(2).

position(train(1), section(16), west).
position(train(2), section(10), east).
```

```
% RULE 1: Make Reservations
{ reserved(train(Z), section(D)) } :-
    position(train(Z), section(S), 0),
    connected(section(S), section(D), 0).
```

```
% RULE 2: Reservation into bidirectional requires onward
   reservation
reserved(train(Z), section(S)) :-
    reserved(train(Z), section(R)),
    position(train(Z), _, 0),
    connected(section(R), section(S), 0),
    bidirectional(section(S)).
```

```
% RULE 3: Trains have to queue up
{ in_front(train(Z), train(Z2), section(S)) : train(Z) !=
   train(Z2) } :-
    reserved(train(Z), section(S)),
    reserved(train(Z2), section(S)),
    train(Z) != train(Z2).
```

There are additional constraints that ensure "in_front" does not close a circle and that there is always an order for every track with multiple reservations.

The main advantage of DRR over MCA is its simplicity and flexibility.

Due to simplified assumptions in how DRR handles secondary consequences it is possible for deadlocks to still occur!

The paper states the likelihood being in “very rare situations”.

Deadlocks and Simulation Systems

- Deadlocks are closed circles of trains waiting on one another
- These can happen especially in networks with bidirectional operation of tracks

---

To analyze deadlock behavior further we looked at them in the context of *synchronous* railway simulation systems.

- In asynchronous simulation systems, scheduling and operation are separated
  - Scheduling creates deadlock free timetables
  - Rescheduling during operation preserves deadlock free property
  - Hence no deadlock from asynchronous simulation systems
- Synchronous simulation systems can exhibit deadlocks as they are not meant to create a deadlock free schedule in the first place

MCA and DRR

To address deadlocks not being detected in synchronous simulations two approaches were presented. The first is Movement Consequence Analysis (MCA).

In MCA we look at the necessary consequences that result from a train moving to a given section. These consequences are either one of two *primary* consequences or *secondary* consequences.

Aiming to address some of MCA's shortcomings, Dynamic Route Reservation(DRR) simplifies some of the underlying assumptions.

In DRR, the route a train takes is reserved multiple sections at a time in advance. This is done in adherence to a set of rules, which *mostly* prevent the occurrence of deadlocks.

 J. Pachl, "Avoiding deadlocks in synchronous railway simulations," 2007.

Questions?